

Entornos de desarrollo

UNIDAD 3

3. Diseño y realización de pruebas.....

Diseño y realización de pruebas

Objetivos

- Comprender la filosofía de las pruebas del software.
- Emplear las estrategias adecuadas para la realización de pruebas a lo largo del ciclo de vida del software.
- Conocer y aplicar técnicas de diseño de casos de prueba de caja blanca y de caja negra.
- Conocer de qué manera se documentan las pruebas.
- Utilizar herramientas de depuración en un entorno de desarrollo.
- Realizar pruebas automáticas en un entorno de desarrollo.

Contenidos

- 3.1. Filosofía de las pruebas del software
- 3.2. Estrategia de pruebas del software
- 3.3. Técnicas de diseño de casos de prueba
- 3.4. Documentación de las pruebas
- 3.5. Herramientas de depuración
- 3.6. Pruebas automáticas

Introducción

Cuando un equipo de desarrollo acepta el encargo de crear una aplicación Informática, desea que el resultado de su trabajo sea de calidad, esto es, que la aplicación responda a las necesidades del cliente y que tenga el menor número posible de errores. El objetivo de la etapa de pruebas, posterior a la programación, es descubrir dichos defectos antes de que el software sea entregado al cliente.

En esta unidad, se explica cómo abordar esta importante tarea y las diferentes técnicas de diseño de casos de prueba. También se verá de qué manera un entorno de desarrollo puede asistir a las personas que desarrollan programas Informáticos en las tareas de depuración o corrección de errores encontrados en esta fase. Para finalizar, se describe el empleo de una herramienta que sirve para la realización de pruebas unitarias de manera automatizada.

***Una aplicación con errores
NO es una aplicación de calidad***

3.1. Filosofía de las pruebas del software

Las pruebas constituyen una de las tareas del ciclo de vida del software y tienen como objetivo fundamental la detección de defectos que se han cometido inadvertidamente durante el proceso de construcción del software.

*Una aplicación con cero errores
raramente se consigue.
Se trata de alcanzar un **EQUILIBRIO**
entre el tiempo y el esfuerzo
de la realización de las pruebas y
la inexistencia de errores.
“Ningún programador es infalible”*

El objetivo de las pruebas puede percibirse desde dos puntos de vista:

1. Mediante las pruebas se pretende, por un lado, comprobar si se está construyendo el producto correcto, es decir, el que desea el cliente, lo cual se relaciona con el concepto de **validación**.
2. Por otro lado, se trata de comprobar si el producto que se está desarrollando funciona correctamente, es decir, si lo que hace lo realiza de manera correcta. Esto se refiere al concepto de **verificación**.

Se trata de ver:

1. *Que el programa es lo que el cliente necesita*
2. *Que el programa funciona de forma correcta*

Recuerda



El hecho de que se encuentren errores a lo largo de la fase de pruebas de una aplicación no debe entenderse como un fracaso del trabajo previo realizado. Al contrario, se debe considerar como algo positivo el hecho de que se encuentren errores después de terminada la programación, pues esto permite eliminarlos antes de entregar la aplicación y evita que estos sean descubiertos por el cliente. No se debe olvidar que «todos cometemos errores» y, como dice el dicho popular, «rectificar es de sabios».

■ 3.2. Estrategia de pruebas del software

Las pruebas siempre comienzan por los componentes más pequeños, es decir, se empieza probando pequeñas porciones de software y se va incrementando de manera progresiva el alcance de la prueba. Se suelen llevar a cabo cuatro tipos de pruebas secuenciales que se exponen en los siguientes subapartados.

■ ■ 3.2.1. Prueba de unidad

■ ■ 3.2.2. Prueba de integración

■ ■ 3.2.3. Prueba del sistema

■ ■ 3.2.4. Prueba de validación

3.2.1. Prueba de unidad

Se comienza probando módulos individuales

3.2.2. Prueba de integración

El objetivo de estas pruebas, que se realizan a continuación de las pruebas de unidad, es comprobar si las clases de las que consta el software funcionan correctamente cuando cooperan entre ellas con el objetivo de realizar las tareas encomendadas al programa.

*Las
iremos
viendo*

1. Prueba basada en hebra: Integra el conjunto de clases requeridas para responder a una entrada o evento del sistema. Cada hebra se integra y prueba de manera individual.
2. Prueba basada en uso: se comienza probando las clases independientes, esto es, aquellas que usan muy pocas clases de servidor o ninguna. Después de probar estas, se examina la siguiente capa de clases, llamadas clases dependientes, que son utilizadas por las primeras. Esta secuencia de pruebas para las capas de clases dependientes continúa hasta que se construye todo el sistema.

3.2.3. Prueba del sistema

- Las iremos viendo todos**
- **Pruebas de recuperación**, ante tolerancia a fallos.
 - **Pruebas de seguridad**, ante jaqueo externo o de protección de datos.
 - **Pruebas de esfuerzo**, ante saturación del sistema por saturación de recursos.
 - **Pruebas de rendimiento**, en tiempos adecuados y de forma eficiente.
 - **Pruebas de despliegue**, en diferentes plataformas y sistemas operativos.

3.2.4. Prueba de validación

El objetivo de la prueba de validación, también llamada **prueba de aceptación**, es determinar si el software es considerado **válido por parte la persona usuaria** y si está **preparado para su implantación** en el entorno de las personas que lo vayan a usar.

En este tipo de pruebas, **participan activamente aquellas personas que lo van a utilizar**, las cuales, **ayudadas por el equipo de pruebas**, deben ejecutar las pruebas. Esta es la fase final del proceso software, en la que la persona usuaria da su consentimiento para su uso o explotación.

Debe estar acorde con la Especificación de Requisitos de Software (ERS) y es la FASE FINAL.

Esquema de las pruebas del software:



Figura 3.1. Representación del modelo en V, en el que la fase posterior a la programación (las pruebas) se divide en varias subfases que corresponden a las etapas del ciclo de vida en cascada, si bien las etapas de análisis y diseño se dividen cada una de ellas en dos.

■ 3.3. Técnicas de diseño de casos de prueba

1. **Pruebas de caja blanca o pruebas estructurales:** examinan los detalles de cada módulo, para lo que se debe disponer del código fuente. A través de dicho código, se prueban los diferentes caminos, los bucles, las variables, etcétera.
2. **Pruebas de caja negra o pruebas funcionales:** se considera al software como una caja negra que recibe una serie de entradas y proporciona una serie de salidas. El objetivo de estas pruebas es validar los requisitos funcionales.

ACTIVIDAD DICCIONARIO:

Elabora un diccionario con los siguientes términos:

- VALIDACIÓN DE LA APLICACIÓN
- ACEPTACIÓN DE LA APLICACIÓN
- PRUEBA DE UNIDAD
- PRUEBA DE INTEGRACIÓN
- PRUEBA DEL SISTEMA
- PRUEBA DE VALIDACIÓN
- PRUEBA DE ACEPTACIÓN
- ESPECIFICACIÓN DE REQUISITOS DEL SOFTWARE (ERS)
- PRUEBAS DE CAJA BLANCA
- PRUEBAS ESTRUCTURALES
- PRUEBAS DE CAJA NEGRA
- PRUEBAS FUNCIONALES

ACTIVIDAD TIPO TEST:

Haz con tu compañero un test de 10 preguntas de tipo test.

3.3.1. Pruebas estructurales o de caja blanca

El objetivo de las pruebas de caja blanca es crear casos de prueba que permitan revisar detalles internos del software. Así, de estas pruebas se derivan casos de prueba que:

1. Garantizan que todas las rutas independientes dentro de un módulo se revisaron al menos una vez.
2. Revisan todas las decisiones lógicas en sus lados verdadero y falso.
3. Ejecutan todos los bucles en sus fronteras y dentro de sus fronteras operativas.
4. Revisan estructuras de datos internas para garantizar su validez.

Existen diferentes técnicas de diseño de casos de prueba de caja blanca, que se exponen seguidamente.



Prueba del camino básico

Esta técnica también recibe el nombre de **prueba de ruta básica** y fue propuesta por Thomas J. McCabe. Esta técnica permite generar una métrica de la complejidad del módulo probado llamada **complejidad ciclomática de McCabe**, que es un número entero que indica el número de caminos independientes que hay en el código y, en consecuencia, el número de casos de prueba que hay que ejecutar para garantizar que se recorren todas las sentencias del código al menos una vez. Para calcular esta métrica y, en consecuencia, generar los casos de prueba, es necesario representar el código fuente que se va a probar en un **grafo de flujo**.

Para crear un grafo de flujo, se deben llevar a cabo los siguientes pasos:

1. Se asigna un número único a cada sentencia del código y a cada condición. No obstante, si se dispone de varias instrucciones en secuencia, es posible asignar un número único a todas ellas y estas serán tratadas como un bloque. Cada uno de estos elementos (sentencia o grupo de sentencias en secuencia o condición) constituirá un **nodo** del grafo de flujo, que se representará mediante un círculo en cuyo interior aparecerá el número asignado.
2. Se unen los nodos mediante flechas llamadas **aristas** que representan el flujo de control y son similares a las flechas de un diagrama de flujo. Se deben representar las estructuras básicas de programación tal y como se muestra en Figura 3.2.

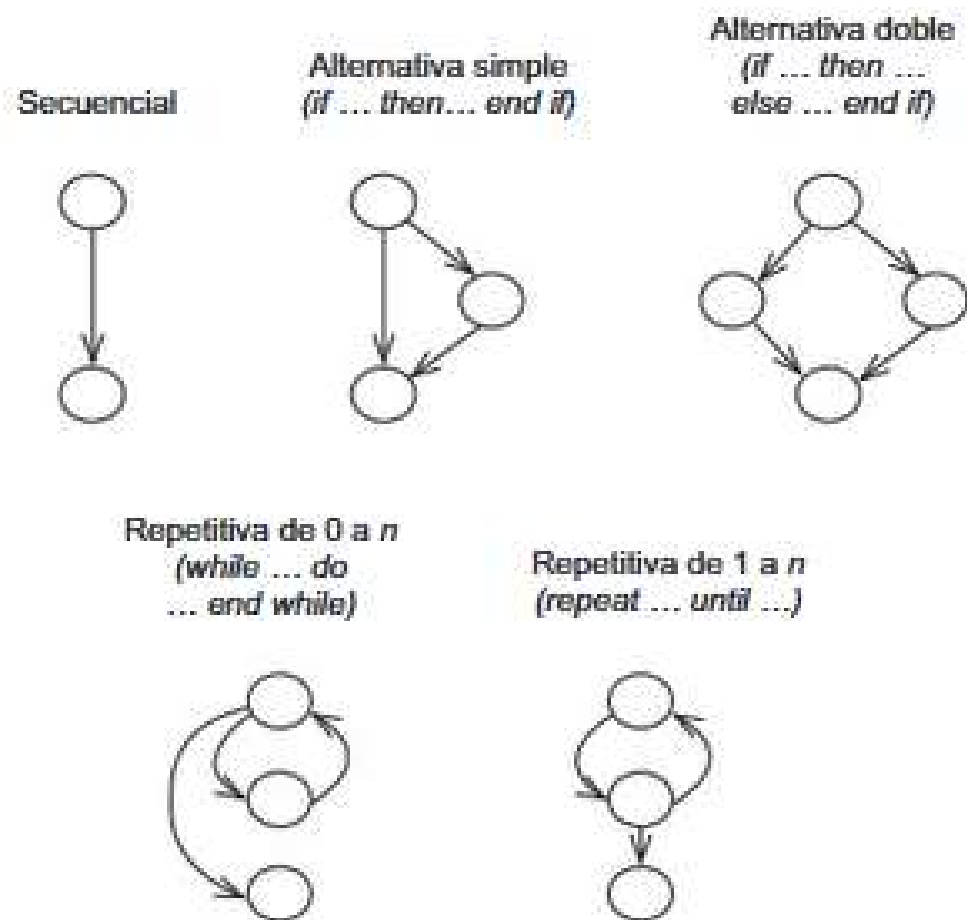


Figura 3.2. Representación de los grafos de flujo para diferentes estructuras de control.

Una vez creado el grafo de flujo, se puede calcular la complejidad ciclomática de McCabe $V(G)$ de cualquiera de las tres formas siguientes:

- **Numero de regiones del grafo** considerando una región como una zona del grafo cerrada. No obstante, también se debe considerar como región la región externa (no cerrada).

- De acuerdo con la fórmula:

$$V(G) = A - N + 2$$

donde **A es el numero de aristas** del grafo, y N , el número de nodos.

- Según la ecuación:

$$V(G) = NP + 1$$

donde **NP es el numero de nodos predicados** del grafo de flujo, es decir, el número de nodos que contienen condiciones.

SE VE MEJOR CON LA
ACTIVIDAD RESUELTA

El **valor $V(G)$ indica el número de caminos independientes del grafo** y, por tanto, el número de **casos de prueba que hay que generar** para garantizar que se han recorrido todas las instrucciones y condiciones del código.

El último paso de este método es generar casos de prueba que recorran cada uno de los caminos, para lo que se deben elegir los datos de entrada que fueren cada uno de esos caminos. Puede ocurrir que alguno de los caminos no pueda recorrerse de manera independiente, sino como parte de otro camino.

Para cada caso de prueba, se deben comparar los resultados obtenidos con los esperados. La no coincidencia indicará la existencia de algún defecto y, en consecuencia, se debe llevar a cabo un proceso de depuración.

Actividad resuelta 3.1

Prueba del camino básico

En este ejercicio se aplica la prueba del camino básico a un método que lee una serie de números hasta que se introduce un cero y muestra la media de los números positivos introducidos y la media de los números negativos.

Solución

La primera tarea que se debe a llevar a cabo es asignar un número entero a cada secuencia de instrucciones y a cada condición:

```
begin
    1  int num = 0, cont_pos=0, cont_neg=0, suma_pos=0, suma_neg=0;
    float media_pos=0, media_neg=0;
    System.out.print("Introduce número: ");
    num=Entrada.entero();
    2  while (num != 0)
    3      if (num > 0)
    4          cont_pos++;
          suma_pos+=num;
    5      else
          cont_neg++;
          suma_neg+=num;
    6      endif;
      System.out.print("Introduce número: ");
      num=Entrada.entero();
    7  endwhile;
    8  if (cont_pos != 0)
    9      media_pos = (float)suma_pos/cont_pos;
      System.out.println("Media de los positivos: "+ media_pos);
    10 endif;
    11 if (cont_neg != 0)
    12     media_neg= (float)suma_neg/cont_neg;
      System.out.println("Media de los negativos: "+ media_neg);
    13 endif;
end;
```

Es posible incluir dentro un mismo nodo varias instrucciones en secuencia, como es el caso de las instrucciones incluidas dentro del nodo 1. También se puede incluir, dentro del mismo nodo, la instrucción que indica el fin de una instrucción condicional y la instrucción o las instrucciones siguientes en secuencia, como se ha hecho para el nodo 6. Asimismo, es posible incluir, dentro de un mismo nodo, la instrucción que indica el fin de una estructura repetitiva junto con las instrucciones anteriores dentro del bucle si son instrucciones en secuencia. Por este motivo, aunque no se ha hecho, también se podrían haber incluido en el mismo nodo las instrucciones asignadas a los nodos 6 y 7.

El siguiente paso es construir el grafo de flujo de acuerdo con las estructuras mostradas en la Figura 3.2. El grafo de flujo quedaría como el que se muestra en la Figura 3.3. En el grafo, se han marcado en rojo los nodos predicados y se ha asignado un número a cada región en color azul.



Una vez que se tiene el grafo de flujo, ya se puede calcular la complejidad ciclomática, $V(G)$, de las tres formas que se ha indicado antes:

$$V(G) = \text{número de regiones} = 5$$

$$V(G) = A - N + 2 = 16 - 13 + 2 = 5$$

$$V(G) = NP + 1 = 4 + 1 = 5$$

A = nº de flechas

N = nº de nodos

NP = nº de nodos predicados (con condiciones)

Como $V(G)$ toma el valor 5, se deduce que hay cinco caminos independientes en el código examinado y, por tanto, el número de casos de prueba que hay que crear para cubrir todas las instrucciones y condiciones del programa es 5. La siguiente tarea es encontrar esos cinco caminos independientes. Para ello, hay que tener en cuenta que cada camino independiente incluye una ruta nueva en relación con los anteriores:

Camino 1: 1-2-8-10-11-13

Camino 2: 1-2-3-4-6-7-2-8-10-11-13

Camino 3: 1-2-3-4-6-7-2-8-9-10-11-13

Camino 4: 1-2-3-5-6-7-2-8-10-11-13

Camino 5: 1-2-3-5-6-7-8-10-11-12-13

Con objeto de probar el camino 1, se debe introducir como primer número el 0 con el fin de que no se entre en el bucle ni una sola vez y se pase directamente del nodo 2 al nodo 8. En este caso, el programa no debe mostrar nada, ya que no se ha introducido ningún número ni positivo, ni negativo.

El camino 2 no se puede probar como tal, ya que al introducir un número positivo (para pasar por el nodo 4), se debería pasar por el nodo 9, que muestra la media de los números positivos. Por consiguiente, este camino debe probarse como parte de otro.

Para probar el camino 3, hay que suministrar al programa, en primer lugar, un número positivo (para pasar por el nodo 4), por ejemplo, el número 8, y un 0 a continuación (para no pasar más por el bucle). El programa deberá indicar que la media de los números positivos es 8. Al probar el camino 3, ya se prueba el camino 2 como parte de él.

El camino 4 tampoco se puede probar como tal, pues al introducir un número negativo (para pasar por el nodo 5), se debe pasar por el nodo 12 para mostrar la media de los números negativos.

Por último, a fin de probar el camino 5, se debe suministrar al programa un número negativo (para pasar por el nodo 5), por ejemplo, el -4, y un 0 a continuación. El programa deberá indicar que la media de los números negativos es -4. Al probar el camino 5, se prueba también el camino 4 como parte de él.

Para cada caso de prueba, se comprueba que el resultado mostrado coincide con el esperado, de forma que la no coincidencia indicará la existencia de algún defecto, por lo que habrá que proceder a detectar y corregir dicho error (mediante un proceso de depuración).



La complejidad ciclomática de McCabe es una métrica de la complejidad del módulo probado. Así, cuando $V(G)$ es mayor que 10, la probabilidad de defectos en el módulo o en el programa crece bastante si dicho valor alto no se debe a sentencias case o similares. En estos casos, es recomendable replantearse el diseño modular obtenido, de modo que sería conveniente dividir el módulo complejo en varios módulos para reducir la complejidad del software.

Prácticas:

Calcular la

COMPLEJIDAD CICLOMÁTICA DE McCABE

de los pseudocódigos de la Unidad 1.

Práctica:

INICIO

LEER NUMERO

IF NUMERO > 0 THEN

ESCRIBE: “EL NÚMERO ES POSITIVO”

ELSE

ESCRIBE: “EL NUMERO ES NEGATIVO”

ENDIF

FIN

Solución:

INICIO

LEER NUMERO ❶

IF NUMERO > 0 THEN ❷

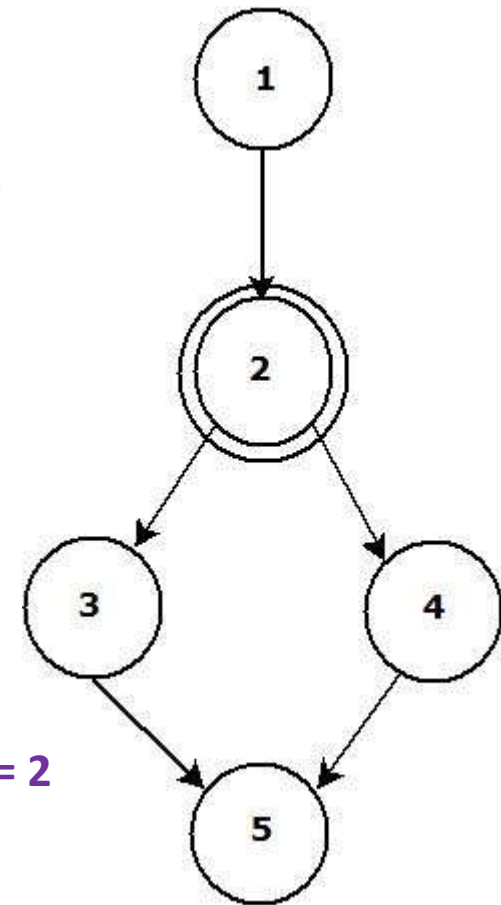
 ESCRIBE: "EL NÚMERO ES POSITIVO" ❸

ELSE

 ESCRIBE: "EL NUMERO ES NEGATIVO" ❹

ENDIF ❺

FIN



$$V(g) = A - N + 2 = 5 - 5 + 2 = 2$$

$$V(g) = Np + 1 = 1 + 1 = 2$$

Práctica:

INICIO

LEER NUMERO1, NUMERO2

IF NUMERO1+NUMERO2 > 10 **THEN**

ESCRIBE: “LA SUMA DE LOS DOS NÚMEROS ES MAYOR QUE 10”

ELSE

IF NUMERO1+NUMERO2 < 10 **THEN**

ESCRIBE: “LA SUMA DE LOS DOS NÚMEROS ES MENOR QUE 10”

ELSE

ESCRIBE: “LA SUMA DE LOS DOS NÚMEROS ES IGUAL A 10”

ENDIF

ENDIF

FIN

Práctica:

INICIO

LEER NUMERO1, NUMERO2 ❶

IF NUMERO1+NUMERO2 > 10 THEN ❷

 ESCRIBE: "LA SUMA DE LOS DOS NÚMEROS ES MAYOR QUE 10" ❸

ELSE

 IF NUMERO1+NUMERO2 < 10 THEN ❹

 ESCRIBE: "LA SUMA DE LOS DOS NÚMEROS ES MENOR QUE 10" ❺

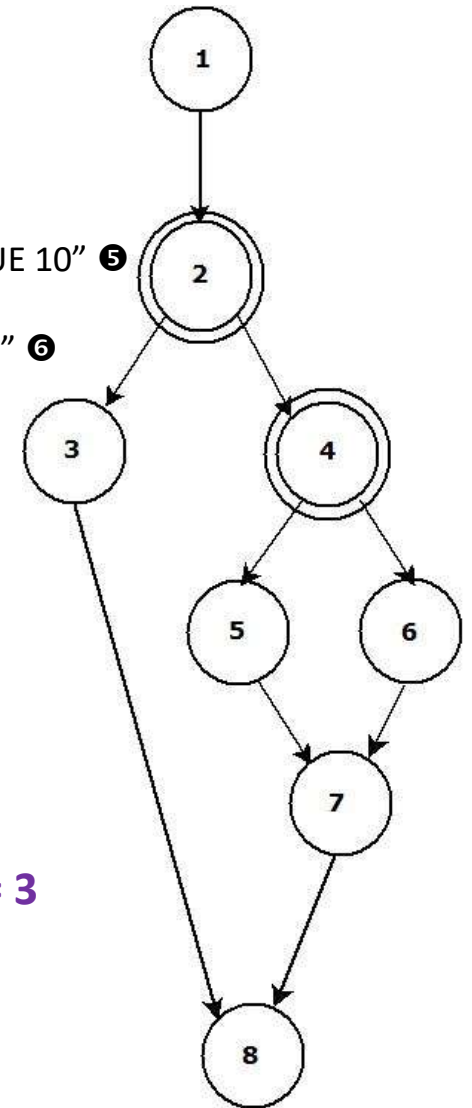
 ELSE

 ESCRIBE: "LA SUMA DE LOS DOS NÚMEROS ES IGUAL A 10" ❻

 ENDIF ❼

ENDIF ❸

FIN



$$V(g) = A - N + 2 = 9 - 8 + 2 = 3$$

$$V(g) = N_p + 1 = 2 + 1 = 3$$

Práctica:

INICIO

CONTADOR = 0

ACUMULADOR = 0

WHILE CONTADOR < 10 **DO**

LEER NUMERO

 CONTADOR = CONTADOR + 1

 ACUMULADOR = ACUMULADOR + NUMERO

ENDDO

ESCRIBE "LA SUMA DE LOS DIEZ NÚMEROS ES"

ESCRIBE ACUMULADOR

FIN

Solución:

INICIO

CONTADOR = 0 ❶

ACUMULADOR = 0 ❶

WHILE CONTADOR < 10 DO ❷

 LEER NUMERO ❸

 CONTADOR = CONTADOR + 1 ❸

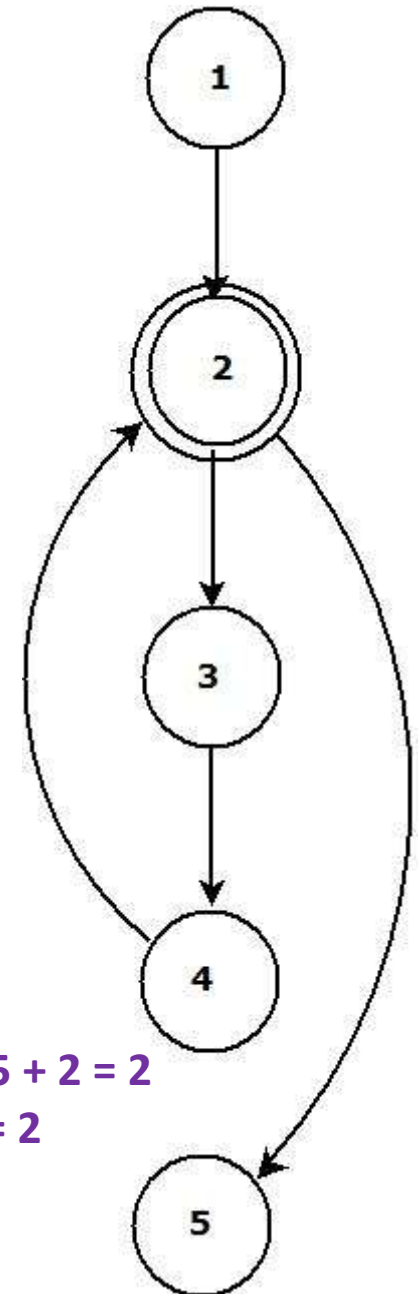
 ACUMULADOR = ACUMULADOR + NUMERO ❸

ENDDO ❹

 ESCRIBE "LA SUMA DE LOS DIEZ NÚMEROS ES" ❺

 ESCRIBE ACUMULADOR ❺

FIN



$$V(g) = A - N + 2 = 5 - 5 + 2 = 2$$

$$V(g) = N_p + 1 = 1 + 1 = 2$$

Actividad propuesta 3.1

Prueba del camino básico

El siguiente programa escrito en Java pide un número entero por teclado e indica si es un número perfecto, es decir, si es igual a la suma de sus divisores propios. Un divisor propio es un entero positivo distinto del número en sí mismo y que divide al número de forma exacta, es decir, sin resto.

Construye el grafo de flujo para este programa y calcula su complejidad ciclomática. Además, indica un conjunto de caminos independientes y, para cada camino, el caso de prueba correspondiente, incluyendo la entrada proporcionada y la salida esperada.

```
1 public static void main(String[] args) {  
2     int num, divisor, sumadivisores;  
3     divisor = 1;  
4     sumadivisores = 0; 1  
5     Scanner entrada = new Scanner(System.in);  
6     System.out.print ("Introduzca un número mayor que 0: ");  
7     num = entrada.nextInt();  
8     while (divisor <= num/2)  
9     {     if (num % divisor == 0)  
10         sumadivisores = sumadivisores + divisor;  
11         divisor++;  
12     }  
13     if (num == sumadivisores)  
14         System.out.println ("El número " + num + " es un número  
15         perfecto");  
16     else  
17         System.out.println ("El número " + num + " no es un número  
18         perfecto");  
19 }
```

Construye el grafo de flujo para este programa y calcula su complejidad ciclomática. Además, indica un conjunto de caminos independientes y, para cada camino, el caso de prueba correspondiente, incluyendo la entrada proporcionada y la salida esperada.

```

1 public static void main(String[] args) {
2     int num, divisor, sumadivisores;
3     divisor = 1;
4     sumadivisores = 0;
5     Scanner entrada = new Scanner(System.in);
6     System.out.print ("Introduzca un número mayor que 0: ");
7     num = entrada.nextInt();
8     while (divisor <= num/2)
9     {   if (num % divisor == 0)
10         sumadivisores = sumadivisores + divisor;
11         divisor++;
12     }
13     if (num == sumadivisores)
14         System.out.println ("El número " + num + " es un número perfecto");
15     else
16         System.out.println ("El número " + num + " no es un número perfecto");
17 }

```

1

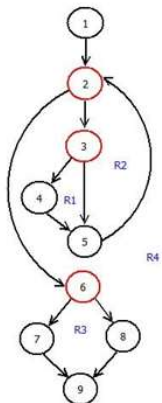
2

3 4 5

6

7

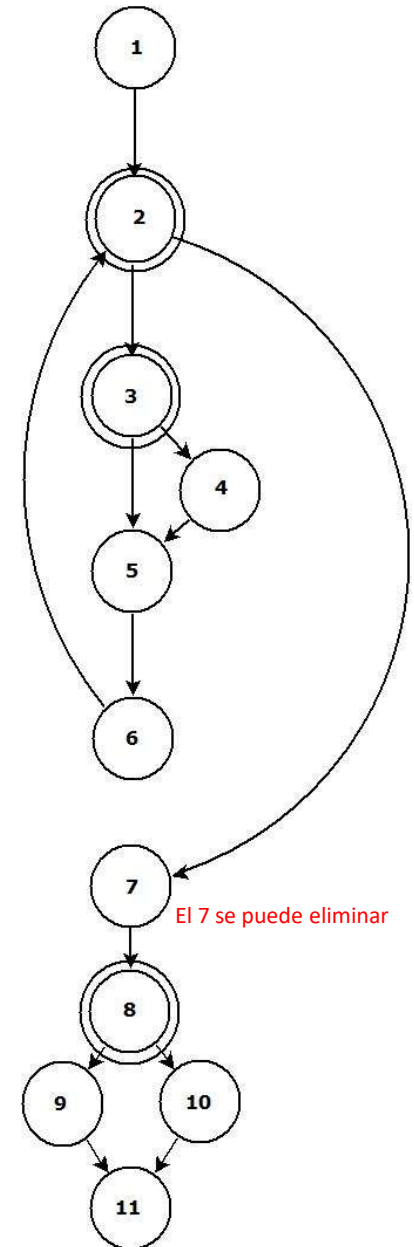
8 9 10 y 1 1



Otra posibilidad.

$$V(g) = A - N + 2 = 13 - 11 + 2 = 4$$

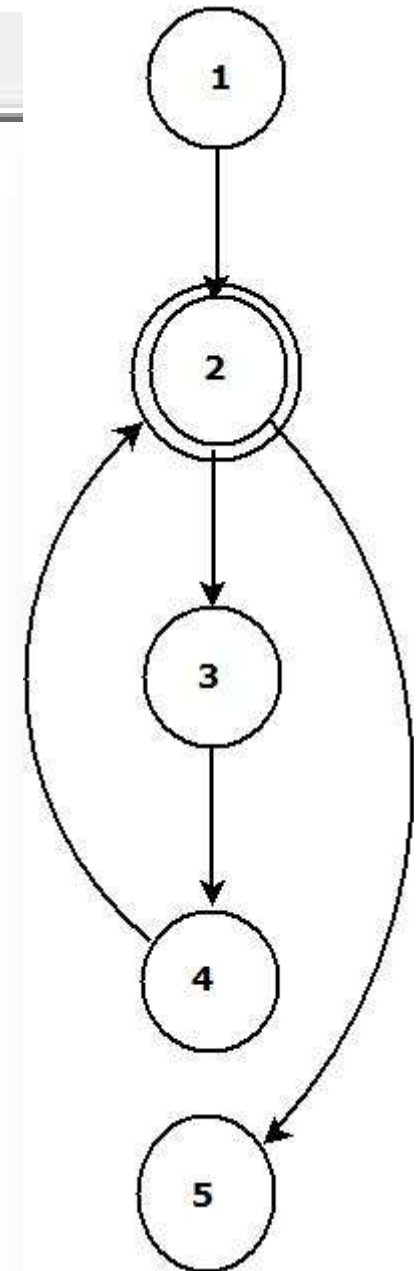
$$V(g) = N_p + 1 = 3 + 1 = 4$$



```
1  Algoritmo sin_titulo
2      contador=0
3      acumulador=0
4      Leer N
5      Mientras contador<N Hacer
6          Leer Edad
7          contador=contador+1
8          acumulador=acumulador+edad
9      Fin Mientras
10     media=acumulador/contador
11     Escribir media
12 FinAlgoritmo
13
```

<sin_titulo> Media edades alumnos.psc X

```
1  Algoritmo sin_titulo
2      contador=0
3      acumulador=0 } ❶
4      Leer N
5      Mientras contador<N Hacer ❷
6          Leer Edad
7          contador=contador+1
8          acumulador=acumulador+edad } ❸
9      Fin Mientras ❹
10     media=acumulador/contador } ❺
11     Escribir media
12 FinAlgoritmo
13
```



Prácticas:

Elaborar el grafo y calcular la

COMPLEJIDAD CICLOMÁTICA DE McCABE

de los pseudocódigos de las 7 diapositivas siguientes.

```
1  Algoritmo sin_titulo
2      contador←0
3      mayores=0
4      Menores=0
5      Mientras contador<10 Hacer
6          Leer numero
7          contador=contador+1
8          Si numero>10 Entonces
9              mayores=mayores+1
10         SiNo
11             menores=menores+1
12         Fin Si
13     Fin Mientras
14     Escribir mayores, menores, iguales
15 FinAlgoritmo
16
```

```
1  Algoritmo sin_titulo
2      contador←0
3      mayores=0
4      Menores=0
5      iguales=0
6      Mientras contador<10 Hacer
7          Leer numero
8          contador=contador+1
9          Si numero>10 Entonces
10             mayores=mayores+1
11         SiNo
12             Si numero<10 Entonces
13                 menores=menores+1
14             SiNo
15                 iguales=iguales+1
16             Fin Si
17         Fin Si
18     Fin Mientras
19     Escribir mayores, menores, iguales
20 FinAlgoritmo
```

```

1  Algoritmo TipoExamen
2      Contador = 0
3      Acumulador = 0
4      Mayor = 0
5      Menor = 100
6      Leer N
7      Mientras contador < N Hacer
8          Leer Numero
9          Contador = contador + 1
10         acumulador = acumulador + numero
11         Si numero > mayor Entonces
12             mayor = numero
13         FinSi
14         Si numero < menor Entonces
15             menor = numero
16         FinSi
17     FinMientras
18     media = acumulador / n
19     Escribir mayor, menor, acumulador, media
20     //
21 FinAlgoritmo
    
```

```
1  Algoritmo sin_titulo
2      contador=0
3      mayores=0
4      menores=0
5      iguales=0
6      pares=0
7      impares=0
8      Mientras contador<10 Hacer
9          Leer numero
10         contador=contador+1
11         Si numero>10 Entonces
12             mayores=mayores+1
13         SiNo
14             Si numero<10 Entonces
15                 menores=menores+1
16             SiNo
17                 iguales=iguales+1
18             Fin Si
19         Fin Si
20         Si numero es par Entonces
21             pares=pares+1
22         SiNo
23             impares=impares+1
24         Fin Si
25     Fin Mientras
26     Escribir mayores, menores, iguales, pares, impares
27 FinAlgoritmo
28
```

INICIO

LEER a , b , c

SI $a > b$ ENTONCES

 SI $a > c$ ENTONCES

 ESCRIBIR a

 SINO

 ESCRIBIR c

 FIN SI

SINO

 SI $b > c$ ENTONCES

 ESCRIBIR b

 SINO

 ESCRIBIR c

 FIN SI

FIN SI

FIN

INICIO

LEER **a**, **b**, c

MIENTRAS **a** $\neq$ **b** HACER

SI **a** > **b** ENTONCES

SI **a** > c ENTONCES

SI **b** > c ENTONCES

ESCRIBIR "**a** mayor"

SINO

ESCRIBIR "c mayor"

FIN SI

SINO

ESCRIBIR "c mayor"

FIN SI

SINO

SI **b** > c ENTONCES

ESCRIBIR "**b** mayor"

SINO

ESCRIBIR "c mayor"

FIN SI

FIN SI

LEER **a**, **b**, c

FIN MIENTRAS

FIN

INICIO

LEER n

$i \leftarrow 1$

MIENTRAS $i \leq n$ HACER

$j \leftarrow 1$

MIENTRAS $j \leq i$ HACER

SI $(i + j) \bmod 3 = 0$ ENTONCE

ESCRIBIR i, j

FIN SI

$j \leftarrow j + 1$

FIN MIENTRAS

$i \leftarrow i + 1$

FIN MIENTRAS

FIN

Prueba de bucles

Los bucles o estructuras repetitivas constituyen una estructura de control fundamental en todo lenguaje de programación. La prueba de bucles se centra exclusivamente en este tipo de estructuras. Los casos de prueba que se deben generar para cada tipo de bucle son los siguientes:

- **Bucles simples:** si el número máximo de iteraciones por el bucle es n , se deben probar los siguientes casos:
 - Saltarse completamente el bucle.
 - Realizar una iteración por el bucle.
 - Realizar dos iteraciones por el bucle.
 - Realizar m iteraciones por el bucle, siendo $m < n$.
 - Realizar $n-1$, n y $n+1$ iteraciones por el bucle.
- **Bucles anidados:** se deben llevar a cabo los siguientes pasos:
 1. Comenzar por el bucle más interno, estableciendo valores mínimos para todos los demás bucles.
 2. Realizar pruebas de bucle simple para el bucle más interno, manteniendo los bucles exteriores en sus valores mínimos.
 3. Trabajar de fuera hacia dentro y probar el siguiente bucle manteniendo los otros bucles exteriores en valores mínimos y los otros bucles interiores en valores típicos.
 4. Continuar hasta que se hayan probado todos los bucles.
- **Bucles concatenados:** si los bucles son independientes unos de otros, se pueden probar empleando la estrategia indicada para los bucles simples. Si los bucles concatenados son dependientes porque el segundo bucle usa el contador del primer bucle como valor inicial para el segundo, se recomienda emplear el enfoque aplicado a los bucles anidados.

Prueba de flujo de datos

Esta técnica de diseño de casos de prueba selecciona caminos de un programa de acuerdo con las ubicaciones de las definiciones y usos de variables en dicho programa.

Aquí es necesario considerar que, en una instrucción, se define una variable si se le asigna valor, mientras que se puede afirmar que esa variable se usa si se consulta o utiliza su valor de algún modo. Así, en una condición, siempre se usan una o varias variables.

Si se asigna el número I a una instrucción, el conjunto DEF para esa instrucción I estará formado por el conjunto de variables que se definen en esa instrucción. Su conjunto USO estará formado por el conjunto de variables cuyo valor se usa en esa instrucción. Esto quedaría ilustrado así:

$$DEF(I) = \{X \mid I \text{ contiene una definición de } X\}$$

$$USO(I) = \{X \mid I \text{ contiene un uso de } X\}$$

La definición de una variable X en la instrucción I está viva en la instrucción I' si hay un camino entre las instrucciones I e I' que no contiene ninguna otra definición de X .

Mediante esta técnica, se encuentran las cadenas **DU** para una variable. Estas cadenas tienen la forma $[X, I, I']$, donde X es una variable, I e I' son instrucciones tal que X está en $DEF(I)$ y en $USO(I')$, y la definición de X en I está viva en I' porque existe algún camino entre I e I' que no incluye ninguna otra definición de X .

Prácticas de PRUEBA DE BUCLES:

Haz un juego de valores de prueba para cada una de las prácticas anteriores.

Elabora una tabla para cada uno de los valores y las veces que recorre el bucle.

Argot técnico



Las **cadenas DU** de esta técnica reciben este nombre porque se trata de **cadenas de definición-uso**, más concretamente, cadenas en las que primero se define una variable y luego se usa esa misma variable sin que haya entre la instrucción donde se define y la instrucción donde se usa ninguna otra definición de la variable.

Debido a que el objetivo es encontrar cadenas DU, esta técnica también recibe el nombre de **prueba DU**.

Esta técnica requiere llevar a cabo los siguientes pasos:

1. Asignar un número a cada instrucción y condición de un programa.
2. Calcular el conjunto *DEF* y *USO* para cada variable que se usa en ese programa.
3. Encontrar las cadenas DU para todas las variables.
4. Generar el número mínimo de casos de prueba que incluyan caminos para todas las cadenas DU.
5. Generar datos de entrada que fuercen cada uno de los caminos encontrados en el paso anterior.

Actividad resuelta 3.2

Prueba de flujo de datos

En este ejercicio se aplica la prueba de flujo de datos al método de la Actividad resuelta 3.1 para el que se obtuvo el grafo de flujo de la Figura 3.3.

Solución

El primer paso ya está hecho porque ya se ha asignado un número a cada instrucción y condición.

Seguidamente, se calculan las cadenas DU solo para la variable *num*. A tal fin, se definen los conjuntos *DEF* y *USO* para esta variable. Como se puede observar, la variable *num* se define en las instrucciones 1 y 6, mientras que se usa en las que tienen asignados los números 2, 3, 4 y 5. Esto se indica del siguiente modo:

$DEF(1) = \{num\}$

$USO(2) = \{num\}$

$USO(3) = \{num\}$

$USO(4) = \{num\}$

$USO(5) = \{num\}$

$DEF(6) = \{num\}$

El siguiente paso es buscar las cadenas DU para esta variable, que son las siguientes numeradas:

$DU[num, 1, 2] (1)$

$DU[num, 1, 3] (2)$

$DU[num, 1, 4] (3)$

$DU[num, 1, 5] (4)$

$DU[num, 6, 2] (5)$

$DU[num, 6, 3] (6)$

$DU[num, 6, 4] (7)$

$DU[num, 6, 5] (8)$

A continuación, se debe encontrar el número mínimo de caminos que incluyan todas las cadenas DU. Estos caminos son:

Actividad propuesta 3.2

Prueba de flujo de datos

Tomando el programa de la Actividad propuesta 3.1, obtén las cadenas DU para la variable *sumadivisores* y encuentra el número mínimo de caminos de prueba que recorren todas estas cadenas.

- Camino 1: 1-2-3-4-6-7-2-3-4-6-7-2-3-5-6-7-2-8-9-10-11-12-13. Este camino cubre las cadenas DU: (1), (2), (3), (5), (6), (7) y (8).
- Camino 2: 1-2-3-5-6-7-2-8-10-11-12-13. Este camino cubre la cadena DU (4).

En este caso, con dos caminos es suficiente para cubrir todas las cadenas DU encontradas. En la primera iteración del bucle del camino 1, se cubren las cadenas DU (1), (2) y (3); en la segunda iteración, se cubren las cadenas DU (5), (6) y (7) y, en la tercera iteración, se cubre la cadena DU (8). Pero, en el camino 1, no es posible cubrir la cadena DU con el número (4), es decir, $DU[num, 1, 5]$ porque por el nodo 1 solo se pasa una vez. En las iteraciones, no se puede pasar por los nodos 4 y 5 a la vez y, para volver a uno de los nodos 4 o 5, hay que pasar por el nodo 6, que contiene otra definición de la variable *num*. Por este motivo, hay que establecer otro camino para cubrir esta cadena DU.

Ya solo quedaría generar los datos de entrada para recorrer cada uno de estos dos caminos. Para el camino 1, habría que introducir en el programa dos números positivos, luego uno negativo y un cero para finalizar. Por ejemplo, si se suministra un 4, un 8 y un -9, el programa nos debe indicar que la media de los números positivos es 6 y la de los negativos, -9. Para forzar el camino 2, habría que introducir en el programa un número negativo y un cero para finalizar. Si, por ejemplo, se proporciona como entrada el número -10 y luego un 0, el programa debe indicar que la media de los números negativos es -10.

■ ■ 3.3.2. Pruebas funcionales o de caja negra

El objetivo de las pruebas de caja negra es crear casos de prueba que permitan revisar todos los requisitos funcionales de un programa. A tal efecto, existen diferentes técnicas de diseño de casos de prueba de caja negra que se exponen a continuación.

■ ■ ■ Particiones o clases de equivalencia

Por medio de esta técnica, se identifican un conjunto de clases de equivalencia o tipos de valores que se deben asignar a los datos de entrada. El primer paso para identificar las clases de equivalencia es determinar las condiciones de entrada del programa. Una condición de entrada puede ser:

- Un valor específico.
- Un rango de valores permitido, por ejemplo, entre 18 y 65.
- Un conjunto de valores permitidos, por ejemplo, 18, 36 y 72.
- Una condición lógica o booleana, por ejemplo, es español o no es español.

Hay una serie de reglas que ayudan a identificar las clases de equivalencia en función del tipo de la condición de entrada:

- Si la condición de entrada es un valor específico, se debe crear una clase válida (con ese valor) y dos no válidas (con dos valores distintos).
- Si la condición de entrada consiste en un rango de valores permitidos (por ejemplo, un número entre 18 y 65), se debe crear una clase válida ($18 \leq \text{número} \leq 65$) y dos no válidas ($\text{número} < 18$ y $\text{número} > 65$).
- Si la condición de entrada consiste en un conjunto de valores permitidos y se sabe que el programa trata de manera diferente cada uno de ellos (por ejemplo, el idioma es inglés, francés o italiano), se identifica una clase válida por cada valor (en este caso, tres valores) y una clase no válida (por ejemplo, alemán).
- Si la condición de entrada consiste en una condición lógica (por ejemplo, la persona debe tener nacionalidad española), se identifica una clase de equivalencia válida (es española) y otra no válida (no es española).
- Si se sabe que algunos elementos de una clase no se tratan de igual forma que el resto de sus elementos, debe dividirse dicha clase de equivalencia en varias clases menores.

Para la creación de los casos de prueba, una vez conocidas estas reglas, se llevan a cabo los siguientes tres pasos:

1. Asignar un número único a cada clase de equivalencia.
2. Crear el número mínimo de casos de prueba que incluyan todas las clases válidas.
3. Crear un caso de prueba por cada clase no válida. El caso de prueba debe incluir una clase no válida, y todas las demás clases deben ser válidas.

Actividad resuelta 3.3

Prueba de particiones o clases de equivalencia

En una aplicación, la persona usuaria debe introducir su edad, que debe ser un número entre 18 y 65; su NIF, que debe constar de 8 números y una letra; y, además, debe indicar si tiene o no nacionalidad española, de forma que tener nacionalidad española es un requisito.

Solución

En la Tabla 3.1, se muestran las condiciones de entrada y, por cada una de ellas, las clases válidas y las no válidas, numerándose cada una de ellas.

Tabla 3.1. Condiciones de entrada para el supuesto, junto con las clases de equivalencia válidas y no válidas de acuerdo con las normas especificadas

Condición de entrada	Clases válidas	Clases no válidas
Edad	$18 \leq \text{edad} \leq 65$ (1)	edad < 18 (2) edad > 65 (3) No es un número (4)
NIF	Una cadena de 9 caracteres compuesta por 8 números y una letra (5)	< 9 caracteres (6) > 9 caracteres (7) Alguno de los 8 primeros caracteres no es un número (8) El último carácter no es una letra (9)
Nacionalidad	Española (10)	No española (11)

En la Tabla 3.2, se muestran los casos de prueba válidos y, en la Tabla 3.3, los no válidos. Por cada caso de prueba, se indica, en la última columna, las clases de equivalencia incluidas. Recuerda que, para el caso de las clases válidas, se debe crear el número mínimo de casos de prueba que incluyan todas las clases válidas; en este caso, se requiere un único caso de prueba.

Tabla 3.2. Caso de prueba con clases de equivalencia válidas

Edad	NIF	Nacionalidad	Clases incluidas
35	32323267G	Española	(1), (5), (10)

Para las clases no válidas, hay que crear un caso de prueba por cada una de ellas. Como se puede observar en la Tabla 3.1, al haber 8 clases de equivalencia no válidas, se deben crear 8 casos de prueba (Tabla 3.3).

Tabla 3.3. Casos de prueba con clases de equivalencia no válidas

Edad	NIF	Nacionalidad	Clases incluidas
16	78787654Z	Española	(2), (5), (10)
73	88788888U	Española	(3), (5), (10)
AB	56837483Y	Española	(4), (5), (10)
45	879847F	Española	(1), (6), (10)
23	6767676762 ^a	Española	(1), (7), (10)
64	TT789009R	Española	(1), (8), (10)
19	569832349	Española	(1), (9), (10)
23	98828282C	No española	(1), (5), (11)

Actividad propuesta 3.3

Prueba de particiones o clases de equivalencia

La consejería de sanidad de una región desea crear una aplicación para posibilitar la solicitud de citas previas de los pacientes con sus médicos. Los datos que deben introducir los pacientes para acceder a esta aplicación son:

- a) Número de tarjeta sanitaria (TIS): debe ser un número entero de 8 dígitos.
- b) Primer apellido: debe ser una cadena de entre 2 y 30 letras, pudiendo incluir algún espacio en blanco.
- c) Año de nacimiento: debe ser un número entero entre 1901 y el año actual.

Crea una tabla de clases de equivalencia. Además, genera los casos de prueba correspondientes usando la técnica de particiones o clases de equivalencia, indicando en cada caso las clases cubiertas.

Tabla 3.1. Clases de equivalencia.

Condición de entrada	Clases Válidas	Clases no válidas
TIS	Un número entero de 8 dígitos (1)	< 8 dígitos (2) > 8 dígitos (3) Alguno de los 8 caracteres no es un número (4) No es un número entero (5)
Primer apellido	Cadena de entre 2 y 30 caracteres con letras y espacios en blanco (6)	< 2 caracteres (7) > 30 caracteres (8) Alguno de los caracteres no es una letra ni un espacio en blanco (9)
Año nacimiento	Número entero entre 1901 y el año actual (10)	Número < 1901 (11) Número > año actual (12) No es un número entero (13) No es un número (14)

Tabla 3.2. Casos de pruebas con clases válidas.

TIS	Primer apellido	Año nacimiento	Clases incluidas
02348723	PÉREZ	1990	(1), (6), (10)

Tabla 3.3. Casos de pruebas con clases no válidas

TIS	Primer apellido	Año nacimiento	Clases incluidas
023487	Pérez	1990	(2), (6), (10)
0234872390	Pérez	1990	(3), (6), (10)
0987TT99	Pérez	1990	(4), (6), (10)
9087,1234	Pérez	1990	(5), (6), (10)
02348723	A	1990	(1), (7), (10)
02348723	Gil de Viedma García-Verdugo Maestro	1990	(1), (8), (10)
02348723	Gil?	1990	(1), (9), (10)
02348723	PÉREZ	1895	(1), (6), (11)
02348723	PÉREZ	2032	(1), (6), (12)
02348723	PÉREZ	199,9	(1), (6), (13)
02348723	PÉREZ	ABCD	(1), (6), (14)

Ejemplo

```
INICIO
LEER NUMERO
IF NUMERO > 0 THEN
    ESCRIBE: "EL NÚMERO ES POSITIVO"
ELSE
    ESCRIBE: "EL NUMERO ES NEGATIVO"
ENDIF
FIN
```

- Si la condición de entrada consiste en una condición lógica (por ejemplo, la persona debe tener nacionalidad española), se identifica una clase de equivalencia válida (es española) y otra no válida (no es española).

Condición de entrada	Clases válidas	Clases no válidas
Numero >= 0	Numero >= 0 (1) Número < 0 (2)	Numero no es un entero (3)

Condición de entrada	Clases válidas	Clases no válidas
Numero ≥ 0	Numero ≥ 0 (1) Número < 0 (2)	Numero no es un entero (3)

Número	Clase
5	(1)
0	(1)
-5	(2)
a	(3)

Práctica:

*Elabora una tabla con las CLASES DE EQUIVALENCIA
y otra con los CASOS DE PRUEBA, para:*

INICIO

LEER NUMERO1, NUMERO2

IF NUMERO1+NUMERO2 > 10 THEN

ESCRIBE: “LA SUMA DE LOS DOS NÚMEROS ES MAYOR QUE 10”

ELSE

IF NUMERO1+NUMERO2 < 10 THEN

ESCRIBE: “LA SUMA DE LOS DOS NÚMEROS ES MENOR QUE 10”

ELSE

ESCRIBE: “LA SUMA DE LOS DOS NÚMEROS ES IGUAL A 10”

ENDIF

ENDIF

FIN

*Hay que tener en cuenta que, en las PRUEBAS DE CAJA NEGRA,
pero os lo pongo para facilitar el aprendizaje. Más adelante no os
pondré el código, solo diré lo que hace el algoritmo, no el código*

Solución:

Condición de entrada	Clases válidas	Clases no válidas
Suma > 10	Suma > 10 (1)	La primera entrada no es un número (2) La segunda entrada no es un número (3)
Suma <= 10	Suma < 10 (4) Suma = 10 (5)	La primera entrada no es un número (2) La segunda entrada no es un número (3)

Números	Clase
0, 0	(5)* Valores límite
5, 7	(1)
5, a	(2)
a, 5	(3)
5, 4	(4)
5, 5	(5)

Práctica:

***Elabora una tabla con las CLASES DE EQUIVALENCIA
y otra con los CASOS DE PRUEBA, para:***

INICIO

CONTADOR = 0

ACUMULADOR = 0

WHILE CONTADOR < 10 DO

LEER NUMERO

CONTADOR = CONTADOR + 1

ACUMULADOR = ACUMULADOR + NUMERO

ENDDO

ESCRIBE “LA SUMA DE LOS DIEZ NÚMEROS ES”

ESCRIBE ACUMULADOR

FIN

Solución:

Condición de entrada	Clases válidas	Clases no válidas
Contador = 10	Entran 10 números (1)	Una de las entradas no es un número (2)

Números	Clase
1, 2, 3, 4, 5, 6, 7, 8, 9, 10	(1)
0, 0, 0, 0, 0, 0, 0, 0, 0, 0	(1)* Valores límite
1, -1, 2, -2, 3, -3, 4, -4, 5, -5	(1)
a, 1, 2, 3, 4, 5, 6, 7, 8, 9	(2)
1, 2, 3, 4, 5, a, 6, 7, 8, 9	(2)
1, 2, 3, 4, 5, 6, 7, 8, 9, a	(2)

Prácticas propuestas:

*Elabora una tabla con las CLASES DE EQUIVALENCIA
y otra con los CASOS DE PRUEBA, para:*

- 1. Un algoritmo que calcule la media de 10 alumnos.*
- 2. Un algoritmo que lea 10 números y cuente cuantos son mayores y cuantos menores o iguales a 10.*
- 3. Un algoritmo que lea 10 números y cuente cuantos son mayores, cuantos menores y cuantos iguales a 10.*
- 4. Un algoritmo que lea n números enteros comprendidos entre 0 y 100 y que obtenga el mayor, el menor, su suma y su media.*
- 5. Un algoritmo que lea n números enteros y cuente cuantos son mayores a 10, cuantos menores a 10, cuantos iguales a 10, cuantos pares y cuantos impares.*
- 6. Un algoritmo que lea 3 números enteros y diga cual es el mayor.*
- 7. Un algoritmo que lea 3 números enteros y los ordene de mayor a menor.*

Conjetura de errores

Esta técnica consiste en generar una lista de errores que suelen cometer quienes desarrollan aplicaciones y de las situaciones que suelen dar lugar a dichos errores, para luego generar un conjunto de casos de prueba basándose en esta lista. Se trata de errores que se cometen comúnmente y no relacionados con aspectos funcionales.

Se muestran a continuación una serie de ejemplos típicos que reflejan el empleo de esta técnica:

- El valor 0 tanto en la entrada como en la salida es una situación propensa a que se cometan errores.
- En situaciones en las que se introduce un número variable de valores, como por ejemplo una lista, conviene considerar los siguientes casos: no introducir ningún valor, introducir un solo valor y un conjunto de valores en el que todos son iguales.
- La usuaria o usuario de la aplicación puede introducir datos erróneos, por ejemplo, que en un campo numérico introduzca algún carácter no numérico.
- La persona encargada de la programación pueda haber interpretado incorrectamente algo en la especificación.

■ ■ 3.3.3. Estrategia de aplicación de las técnicas de diseño de casos de prueba

El proceso de prueba de una aplicación completa se debe ejecutar de acuerdo con los siguientes pasos:

- Se comienza realizando la prueba de unidad de cada clase. Se puede comenzar con la técnica de análisis de valores límite, añadir clases válidas y/o no válidas no contempladas con la técnica de clases de equivalencia y finalizar con la técnica de conjetura de errores.
- Si no se ha alcanzado la cobertura deseada (por ejemplo, el recorrido de todos los posibles caminos a lo largo del código fuente), se deben añadir los casos necesarios mediante técnicas de pruebas de caja blanca (es decir, la prueba del camino básico, la de bucles y la de flujo de datos).
- Se continúa con las pruebas de integración, aunque también se pueden combinar estas con pruebas de unidad. Las pruebas de integración se deben centrar en la comprobación de los mecanismos de interacción entre las diferentes clases y se deben usar preferentemente técnicas de caja blanca. Se debe ir incrementando progresivamente el alcance de estas pruebas hasta abarcar el software completo.
- La prueba del sistema debe emplear casos de prueba de caja negra para comprobar el cumplimiento de los objetivos requeridos por el sistema.
- En la prueba de validación, deben tomar parte las personas usuarias finales y se pueden reutilizar casos de prueba del sistema. La finalización exitosa de estas pruebas supone la aceptación final del software por parte del usuario o usuaria.

Recuerda



Una buena estrategia de prueba debe combinar los dos tipos de técnicas de diseño de casos de prueba estudiadas (pruebas de caja blanca y de caja negra) para lograr la cobertura deseada.

■ 3.4. Documentación de las pruebas

La documentación de las pruebas es conveniente para una buena organización de estas y para asegurar su reutilización. Se deben documentar tanto el diseño de las pruebas como el resultado o ejecución de las pruebas.

En cuanto a la **documentación del diseño de las pruebas**, es conveniente crear los siguientes documentos:

- **Plan de pruebas:** es un documento con la planificación general de las pruebas para la aplicación que se está creando. Se debe indicar el enfoque general de las pruebas, los recursos requeridos, las actividades de pruebas que se van a llevar a cabo, los elementos que se deben probar, el personal responsable y los riesgos asociados.
- **Especificación del diseño de las pruebas:** es un informe que detalla el plan de pruebas porque en él se indica cada uno de los elementos concretos que se van a probar y los procedimientos de prueba. A partir de este documento se generan los siguientes:
 - *Especificaciones de casos de prueba:* por cada prueba que se va a realizar, se deben indicar los datos de entrada que se van a suministrar, los resultados esperados y las dependencias entre casos de prueba.
 - *Especificaciones de procedimientos de prueba:* especifica los pasos para la ejecución de uno o de un conjunto de casos de prueba. En este documento, se deben indicar la lista de casos de prueba correspondientes al procedimiento, los requisitos para la ejecución de los casos de prueba (entorno, personal, etc.) y los pasos del procedimiento.

En cuanto a la **documentación de la ejecución de las pruebas**, esta es importante para la eficacia en la detección y corrección de defectos y para dejar constancia de los resultados de las pruebas. La ejecución de las pruebas toma como entrada las especificaciones de los casos de prueba que se van a usar y las especificaciones de los procedimientos de prueba, y debe generar dos tipos de documentos:

1. **Histórico de pruebas:** registra todos los hechos relevantes ocurridos durante la ejecución: elementos probados, entorno de pruebas, resultados obtenidos, fecha y hora y referencia al informe de incidentes, si es el caso.
2. **Informe de incidentes:** registra cada incidente ocurrido durante la prueba que requiera una posterior investigación.

La ejecución de las pruebas correspondiente a cada especificación del diseño de las pruebas genera asimismo un **informe resumen de pruebas**, que incluye: resumen de la evaluación de los elementos probados, variaciones del software como resultado de las pruebas, valoración de la cobertura de la prueba, resumen de los resultados obtenidos y resumen de las actividades de prueba (incluyendo el consumo de recursos).

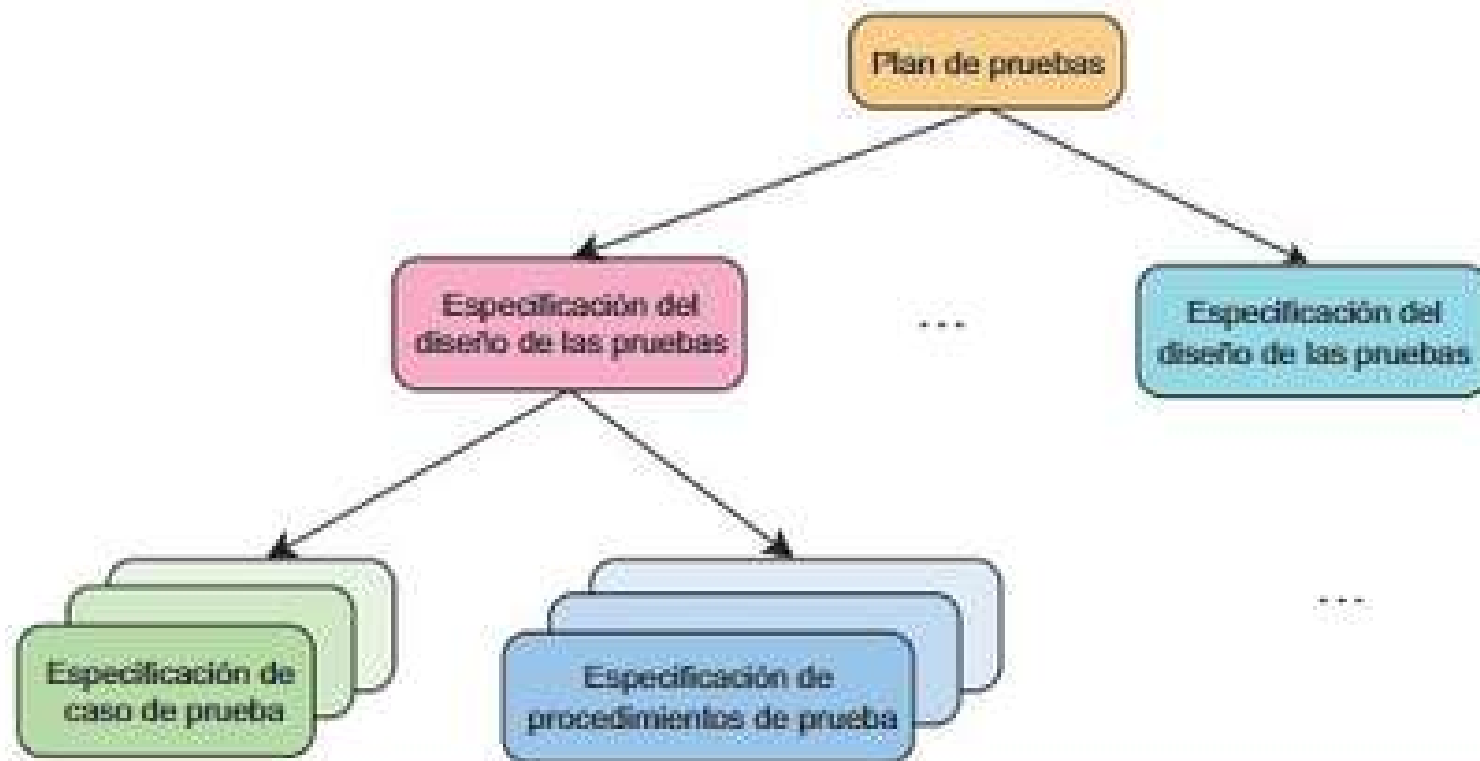


Figura 3.4. El plan de pruebas es un documento único para una aplicación con indicaciones generales acerca de las pruebas. Este se desglosa, por cada elemento que se va a probar, en varias especificaciones del diseño de las pruebas. Cada uno de estos documentos contiene además varias especificaciones de casos de prueba y varias especificaciones de procedimientos de pruebas.

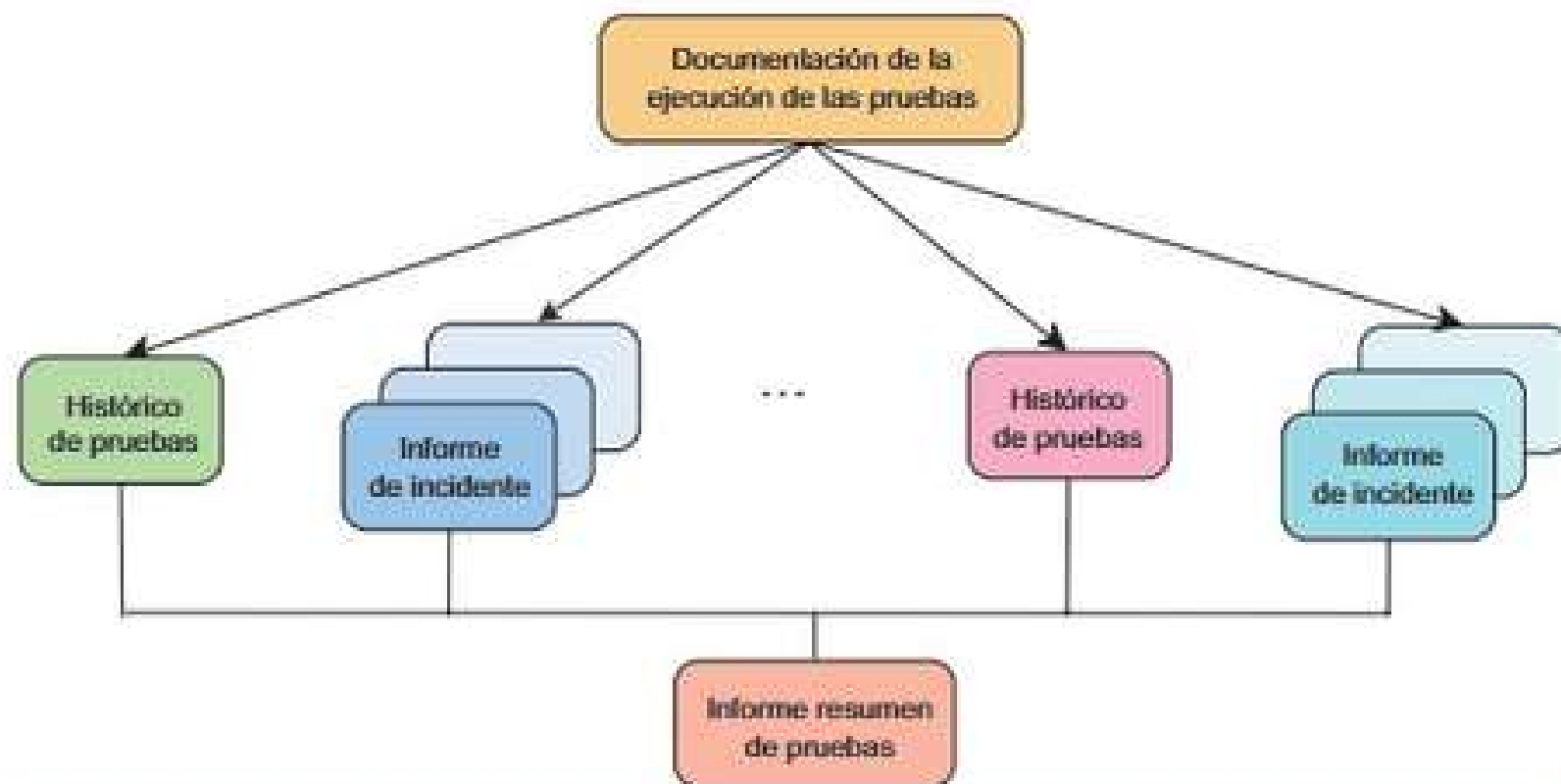


Figura 3.5. Se parte de una especificación del diseño de las pruebas y, por cada ejecución de estas, a partir de una serie de casos de pruebas y sus procedimientos, se genera un histórico de pruebas con todos los hechos relevantes ocurridos durante esas pruebas y un informe por cada incidente ocurrido. Todo ello se recoge en el informe resumen de pruebas.

Prácticas de DOCUMENTACIÓN DE PRUEBAS:

***Elabora un documento de planificación de pruebas
llamado “Plan de pruebas_nombre”***

(Puedes usar modelos de internet o ChatGpt)

***Elabora un documento de ejecución de pruebas
llamado “Informe de ejecución de pruebas_nombre”***

(Puedes usar modelos de internet o ChatGpt)

Elabora los documentos de prueba para:

- 1. Un algoritmo que calcule la media de 10 alumnos.*
- 2. Un algoritmo que lea 10 números y cuente cuantos son mayores y cuantos menores o iguales a 10.*
- 3. Un algoritmo que lea 10 números y cuente cuantos son mayores, cuantos menores y cuantos iguales a 10.*
- 4. Un algoritmo que lea n números enteros comprendidos entre 0 y 100 y que obtenga el mayor, el menor, su suma y su media.*
- 5. Un algoritmo que lea n números enteros y cuente cuantos son mayores a 10, cuantos menores a 10, cuantos iguales a 10, cuantos pares y cuantos impares.*
- 6. Un algoritmo que lea 3 números enteros y diga cual es el mayor.*
- 7. Un algoritmo que lea 3 números enteros y los ordene de mayor a menor.*

Actividad para la plataforma.